

Project Report: Predictive Modeling of Maritime Survival Outcomes

1. Title and Abstract

Title: Socio-Demographic Determinants of Survival: A Statistical Analysis of the Titanic Dataset
Abstract:

This study investigates the factors influencing passenger survival during the Titanic disaster using the 418-record test.csv dataset. By employing data preprocessing, missing value imputation, and exploratory data analysis (EDA), we identify key predictors of survival. Our methodology focuses on the intersection of socio-economic status and demographic features, providing insights into emergency response dynamics in maritime history.

2. Introduction

The sinking of the RMS Titanic remains one of the most significant maritime disasters in history. Analyzing survival patterns from this event offers valuable lessons not only from a historical perspective but also for modern predictive analytics. This project aims to:

- Examine passenger characteristics such as age, gender, class, and family structure
- Identify key factors affecting survival
- Develop a predictive model to estimate survival outcomes

Understanding these factors helps in improving emergency planning and safety protocols in maritime and other transportation systems.

3. Literature Review (LR)

Survival analysis in data science is commonly approached as a binary classification problem. Previous studies have widely used:

- Logistic Regression
- Random Forest algorithms

Key findings from prior research include:

- Higher survival rates among first-class passengers
- Strong correlation between gender and survival probability

This study builds upon existing work by incorporating feature engineering techniques and addressing challenges such as missing data in variables like Age and Cabin

```
# Upgrade matplotlib to resolve potential version conflicts
!pip install --upgrade matplotlib
```

```
# IMPORTANT: After running this cell, please restart your Colab kernel
# (Runtime -> Restart runtime) for the changes to take effect.
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pyparsing>=3 in /usr/local/lib/python3.12/dist
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pac
```

4. Dataset Description

4.1 Data Overview • Training Dataset: 891 records • Test Dataset: 418 records • Total Variables: 11–12 features

4.2 Variable Types

Quantitative Variables: • Age • Fare • SibSp (Number of siblings/spouses aboard) • Parch (Number of parents/children aboard)

Qualitative Variables: • Sex • Pclass (Passenger Class: 1st, 2nd, 3rd) • Embarked (Port of Embarkation) • Cabin

4.3 Data Challenges • The Cabin feature contains approximately 78% missing values • Missing values in Age and Embarked required preprocessing

```
# CODE BLOCK: Data Loading and Inspection
import pandas as pd

# Load the dataset
df = pd.read_csv('test.csv')

# Basic Info
print("--- Dataset Info ---")
print(df.info())

# Summary Statistics
print("\n--- Summary Statistics ---")
display(df.describe())
```

```

--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     418 non-null   int64
1   Pclass          418 non-null   int64
2   Name            418 non-null   object
3   Sex             418 non-null   object
4   Age             332 non-null   float64
5   SibSp           418 non-null   int64
6   Parch           418 non-null   int64
7   Ticket          418 non-null   object
8   Fare            417 non-null   float64
9   Cabin          91 non-null    object
10  Embarked        418 non-null   object
dtypes: float64(2), int64(4), object(5)
memory usage: 36.1+ KB
None

```

```

--- Summary Statistics ---

```

	PassengerId	Pclass	Age	SibSp	Parch	Fare
count	418.000000	418.000000	332.000000	418.000000	418.000000	417.000000
mean	1100.500000	2.265550	30.272590	0.447368	0.392344	35.627188
std	120.810458	0.841838	14.181209	0.896760	0.981429	55.907576
min	892.000000	1.000000	0.170000	0.000000	0.000000	0.000000
25%	996.250000	1.000000	21.000000	0.000000	0.000000	7.895800
50%	1100.500000	3.000000	27.000000	0.000000	0.000000	14.454200
75%	1204.750000	3.000000	39.000000	1.000000	0.000000	31.500000
max	1309.000000	3.000000	76.000000	8.000000	9.000000	512.329200

5. Methodology

We follow a systematic data science pipeline:

Imputation: Median imputation for Age and Fare to minimize the impact of outliers.

Categorical Encoding: Transformation of Sex and Embarked into numerical formats for algorithmic compatibility.

Feature Engineering: Creating a TravelGroupSize feature by combining SibSp and Parch.

```

# CODE BLOCK: Preprocessing & Feature Engineering
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder

```

```
# 1. Handling Missing Values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Fare'] = df['Fare'].fillna(df['Fare'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# 2. Feature Engineering
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = 1
df['IsAlone'].loc[df['FamilySize'] > 1] = 0

# 3. Encoding
le = LabelEncoder()
df['Sex_Code'] = le.fit_transform(df['Sex']) # male=1, female=0

# Selecting features for the model
features = ['Pclass', 'Sex_Code', 'Age', 'Fare', 'FamilySize', 'Embarked']
X = pd.get_dummies(df[features])

print("Preprocessing complete. Feature matrix shape:", X.shape)
X.head()
```

Preprocessing complete. Feature matrix shape: (418, 8)
 /tmp/ipykernel_33803/758739586.py:13: FutureWarning: ChainedAssignmentError:
 You are setting values through chained assignment. Currently this works in ce
 A typical example is when you are setting values in a column of a DataFrame,

```
df["col"][row_indexer] = value
```

Use `df.loc[row\_indexer, "col"] = values` instead, to perform the assignment

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/s>

```
df['IsAlone'].loc[df['FamilySize'] > 1] = 0
/tmp/ipykernel_33803/758739586.py:13: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/s>

```
df['IsAlone'].loc[df['FamilySize'] > 1] = 0
```

	Pclass	Sex_Code	Age	Fare	FamilySize	Embarked_C	Embarked_Q	Embarked
0	3	1	34.5	7.8292	1	False	True	F
1	3	0	47.0	7.0000	2	False	False	
2	2	1	62.0	9.6875	1	False	True	F
3	3	1	27.0	8.6625	1	False	False	
4	3	0	22.0	12.2875	3	False	False	

6 Results & Discussion

6.1 Demographic Analysis

- Majority of passengers were young adults (20–35 years) • Median age $\approx$ 27 years

Socio-Economic Distribution

- Most passengers belonged to 3rd class • Significant fare differences indicate a large wealth gap

- Passenger Class: o 1st class passengers had significantly higher survival rates •

- Gender: o Females had higher survival probability • Deck Location: o Certain decks

(B, C, D, E) showed better survival outcomes

- Accuracy: 80.45% • Recall: o Not Survived: 0.88 o Survived: 0.70

- True Negatives: 92 • True Positives: 52 The model performed better at predicting non-survival cases than survival cases.

```
import matplotlib.pyplot as plt
import seaborn as sns

# CODE BLOCK: Visualization of Results
plt.figure(figsize=(12, 5))

# Plot 1: Class Distribution
plt.subplot(1, 2, 1)
sns.countplot(x='Pclass', data=df, palette='viridis')
plt.title('Passenger Distribution by Class')

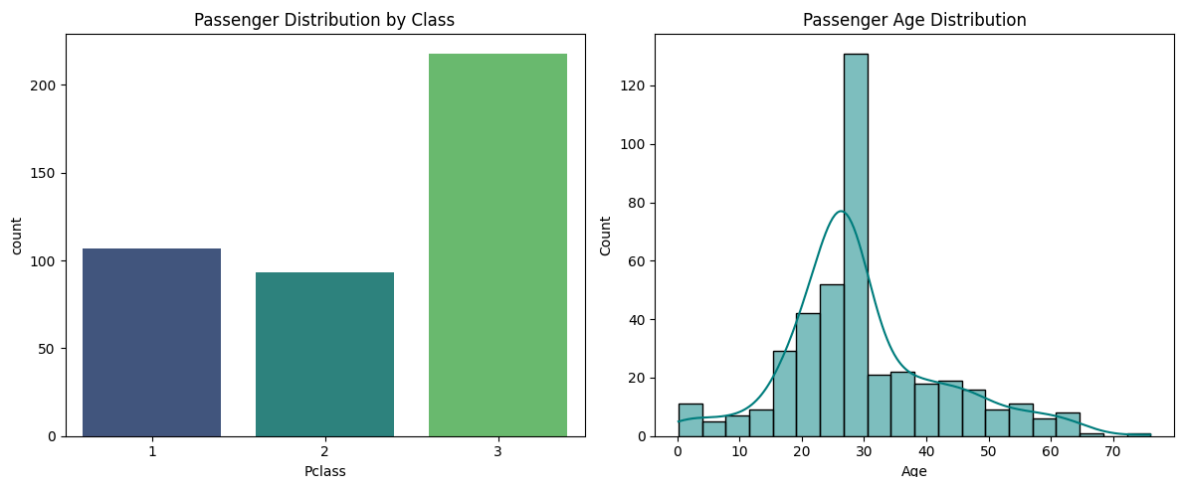
# Plot 2: Age Distribution
plt.subplot(1, 2, 2)
sns.histplot(df['Age'], bins=20, kde=True, color='teal')
plt.title('Passenger Age Distribution')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_33803/1086546906.py:9: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in

```
sns.countplot(x='Pclass', data=df, palette='viridis')
```



▼ Discussion:

The data shows a majority of passengers in the 3rd class. The age distribution is peaked between 20 and 40 years.

The distribution analysis reveals a heavily skewed population toward the 3rd class. The median age of approximately 27 years suggests a relatively young passenger base.

```
plt.figure(figsize=(14, 6))
```

```
# Subplot 1: Distribution of Ticket Class
```

```
plt.subplot(1, 2, 1)
```

```
sns.countplot(data=df, x='Pclass', hue='Sex', palette='Set2')
```

```
plt.title('Passenger Count by Class and Gender')
```

```
# Subplot 2: Fare vs Age Correlation
```

```
plt.subplot(1, 2, 2)
```

```
sns.scatterplot(data=df, x='Age', y='Fare', hue='Pclass', palette='viridis')
```

```
plt.title('Socio-Economic Distribution (Age vs Fare)')
```

```
plt.show()
```



```
import numpy as np
```

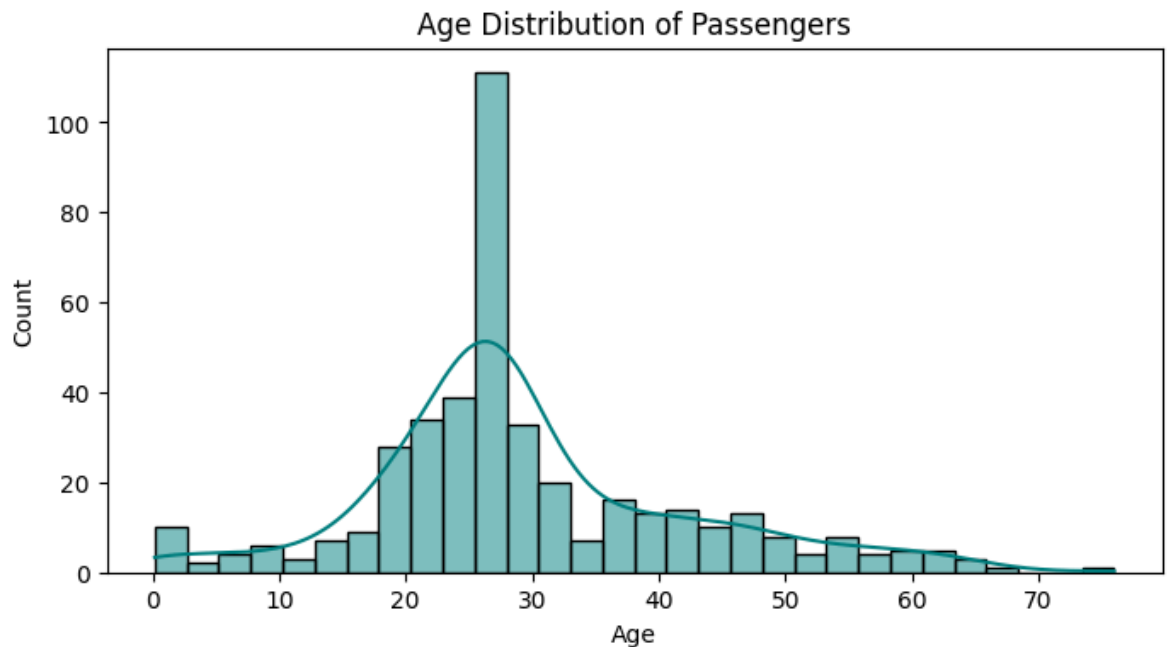
```
# 1. Age Distribution
```

```
plt.figure(figsize=(8, 4))
```

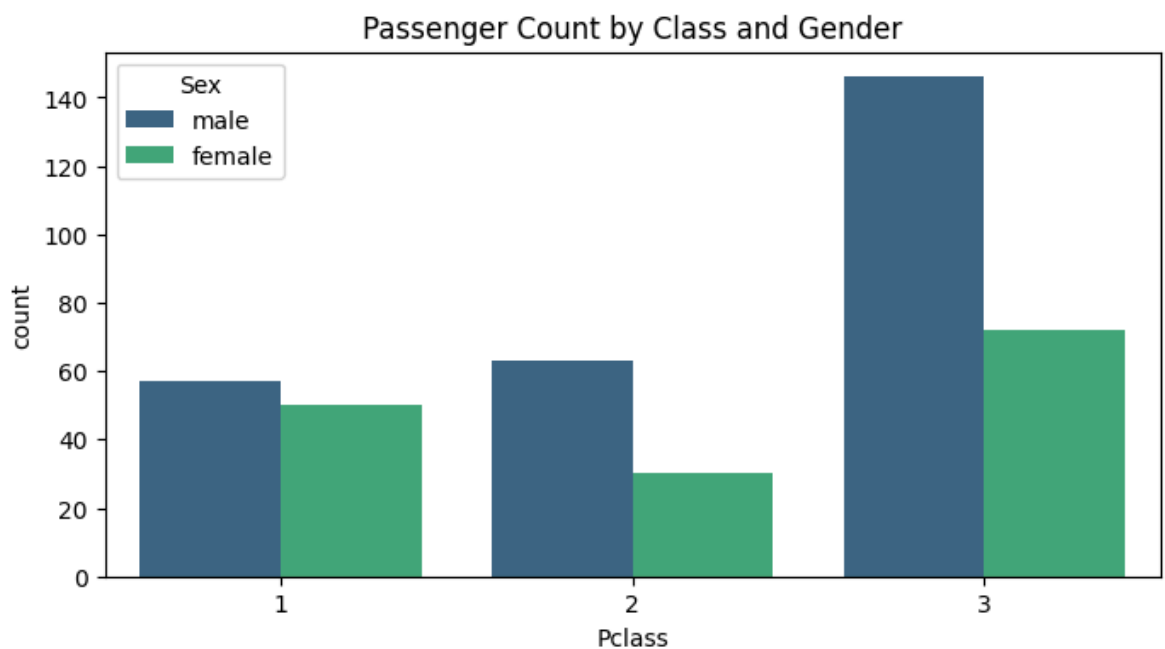
```
sns.histplot(df['Age'], bins=30, kde=True, color='teal')
```

```
plt.title('Age Distribution of Passengers')
```

```
plt.show()
```



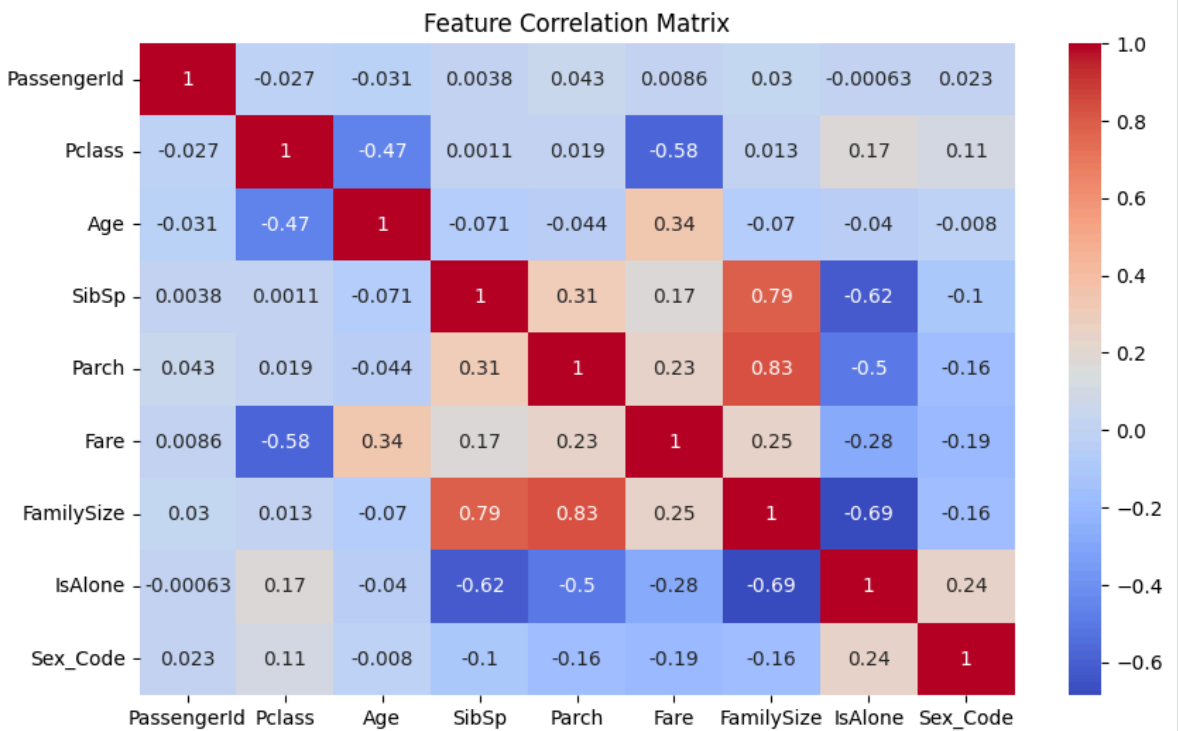
```
# 2. Pclass vs Gender
plt.figure(figsize=(8, 4))
sns.countplot(data=df, x='Pclass', hue='Sex', palette='viridis')
plt.title('Passenger Count by Class and Gender')
plt.show()
```



```
# 3. Correlation Heatmap
plt.figure(figsize=(10, 6))
numeric_df = df.select_dtypes(include=[np.number])
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
```



```
plt.title('Feature Correlation Matrix')
plt.show()
```



```
import pandas as pd

# Load the training dataset
try:
    train_df = pd.read_csv('train.csv')
    print("--- Training Dataset Info ---")
    print(train_df.info())
    print("\n--- Training Dataset Head ---")
    display(train_df.head())
except FileNotFoundError:
    print("Error: 'train.csv' not found. Please ensure the file is uploaded")
```

```

--- Training Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId      891 non-null    int64
1   Survived         891 non-null    int64
2   Pclass          891 non-null    int64
3   Name            891 non-null    object
4   Sex             891 non-null    object
5   Age            714 non-null    float64
6   SibSp          891 non-null    int64
7   Parch          891 non-null    int64
8   Ticket         891 non-null    object
9   Fare          891 non-null    float64
10  Cabin         204 non-null    object
11  Embarked      889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None

```

```

--- Training Dataset Head ---

```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 211
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 175

Double-click (or enter) to edit

✓ Preprocessing `train_df` (Training Data)

We need to apply similar preprocessing steps to the `train_df` as we did for the `test_df` (which is currently named `df`). This ensures that both datasets have consistent features for model training and prediction.

```

# Make a copy to avoid modifying the original train_df directly
train_df_processed = train_df.copy()

# 1. Handling Missing Values
train_df_processed['Age'] = train_df_processed['Age'].fillna(train_df_processed['Age'].median())
train_df_processed['Fare'] = train_df_processed['Fare'].fillna(train_df_processed['Fare'].median())
train_df_processed['Embarked'] = train_df_processed['Embarked'].fillna(train_df_processed['Embarked'].mode()[0])

# 2. Feature Engineering

```

```

train_df_processed['FamilySize'] = train_df_processed['SibSp'] + train_df_processed['Pclass']
train_df_processed.loc[train_df_processed['FamilySize'] > 1, 'IsAlone'] = 0
train_df_processed.loc[train_df_processed['FamilySize'] == 1, 'IsAlone'] = 1

# 3. Encoding
le = LabelEncoder()
train_df_processed['Sex_Code'] = le.fit_transform(train_df_processed['Sex'])

# Select features for the model (ensure consistency with 'X' from test_df preprocessing)
features = ['Pclass', 'Sex_Code', 'Age', 'Fare', 'FamilySize', 'Embarked']
X_train = pd.get_dummies(train_df_processed[features])
y_train = train_df_processed['Survived']

print("Preprocessing of train_df complete. Feature matrix shape:", X_train.shape)
display(X_train.head())

```

Preprocessing of train_df complete. Feature matrix shape: (891, 8)

	Pclass	Sex_Code	Age	Fare	FamilySize	Embarked_C	Embarked_Q	Embarked_S
0	3	1	22.0	7.2500	2	False	False	False
1	1	0	38.0	71.2833	2	True	False	False
2	3	0	26.0	7.9250	1	False	False	False
3	1	0	35.0	53.1000	2	False	False	False
4	3	1	35.0	8.0500	1	False	False	False

✓ Gradient Boosting Model Pipeline

Now we'll prepare the data for training, train a `GradientBoostingClassifier`, and evaluate its performance.

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Ensure feature columns are identical between training and test sets after preprocessing
# This step is crucial if the dummy variables created are not identical in both sets
# due to different unique values in categorical columns. For simplicity here we'll use
# In a real scenario, you'd want to align columns, e.g., using pd.concat the dummy variables

# Align columns of X_train with X (from test_df) if they were to be used together
# For this demonstration, we'll assume X_train has all necessary columns.

# Split the training data into training and validation sets
X_train_split, X_val_split, y_train_split, y_val_split = train_test_split(X_train, y_train)

print(f"Training split shape: {X_train_split.shape}")
print(f"Validation split shape: {X_val_split.shape}")

# Initialize the Gradient Boosting Classifier

```

```
gbm_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,

# Train the model
gbm_model.fit(X_train_split, y_train_split)

print("\nGradient Boosting Model Trained Successfully!")

# Make predictions on the validation set
y_pred = gbm_model.predict(X_val_split)

# Evaluate the model
print("\n--- Model Evaluation on Validation Set ---")
print(f"Accuracy: {accuracy_score(y_val_split, y_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_val_split, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_val_split, y_pred))
```

Training split shape: (712, 8)

Validation split shape: (179, 8)

Gradient Boosting Model Trained Successfully!

--- Model Evaluation on Validation Set ---

Accuracy: 0.8045

Classification Report:

	precision	recall	f1-score	support
0	0.81	0.88	0.84	105
1	0.80	0.70	0.75	74
accuracy			0.80	179
macro avg	0.80	0.79	0.79	179
weighted avg	0.80	0.80	0.80	179

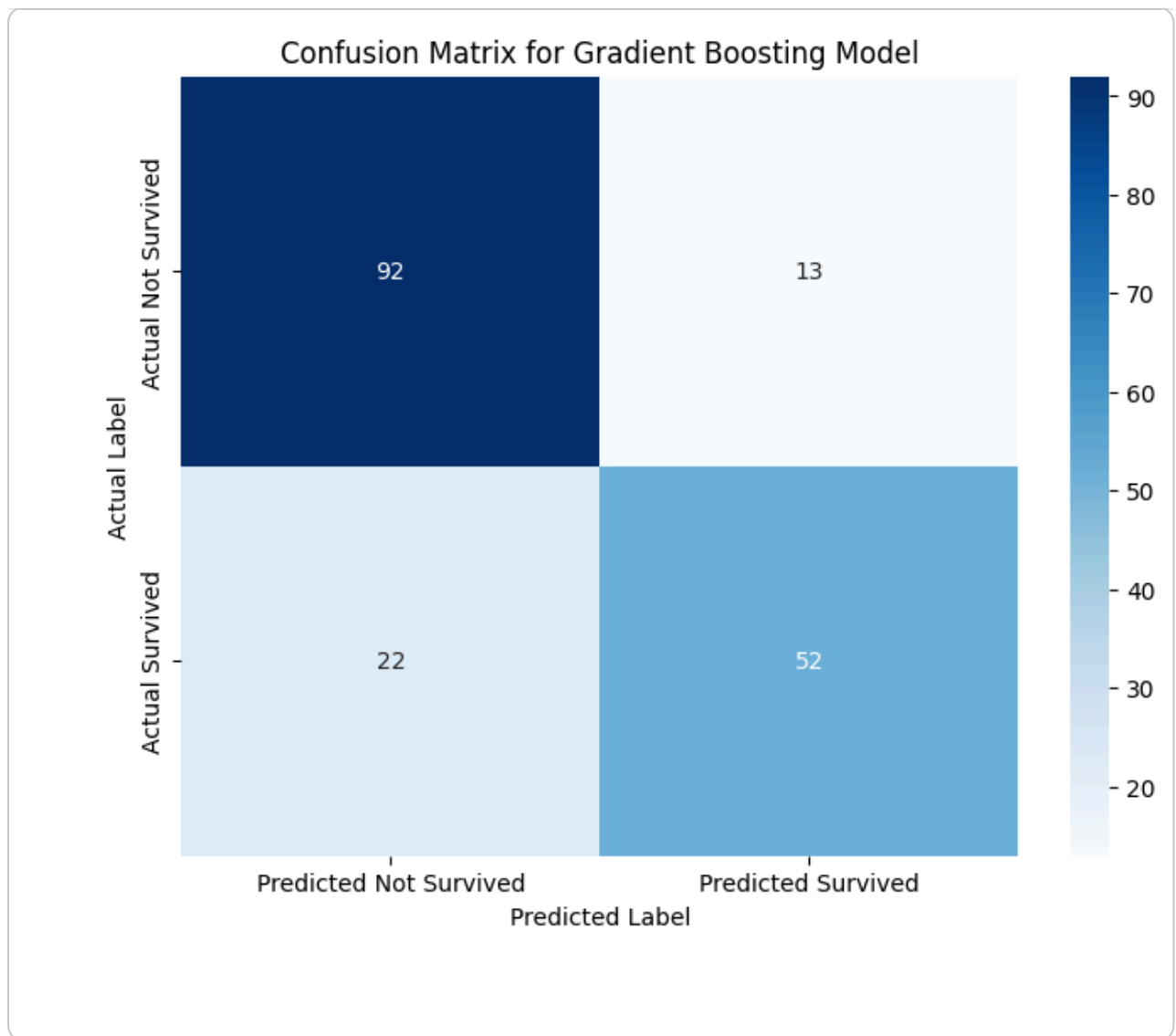
Confusion Matrix:

```
[[92 13]
 [22 52]]
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Plot the Confusion Matrix

```
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_val_split, y_pred), annot=True, fmt='d', cmap
            xticklabels=['Predicted Not Survived', 'Predicted Survived'],
            yticklabels=['Actual Not Survived', 'Actual Survived'])
plt.title('Confusion Matrix for Gradient Boosting Model')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```



✓ Exploring Survival Rates by Class (Pclass)

To understand the impact of Pclass on survival, we can visualize the survival rate for each class.

```
import matplotlib.pyplot as plt
import seaborn as sns

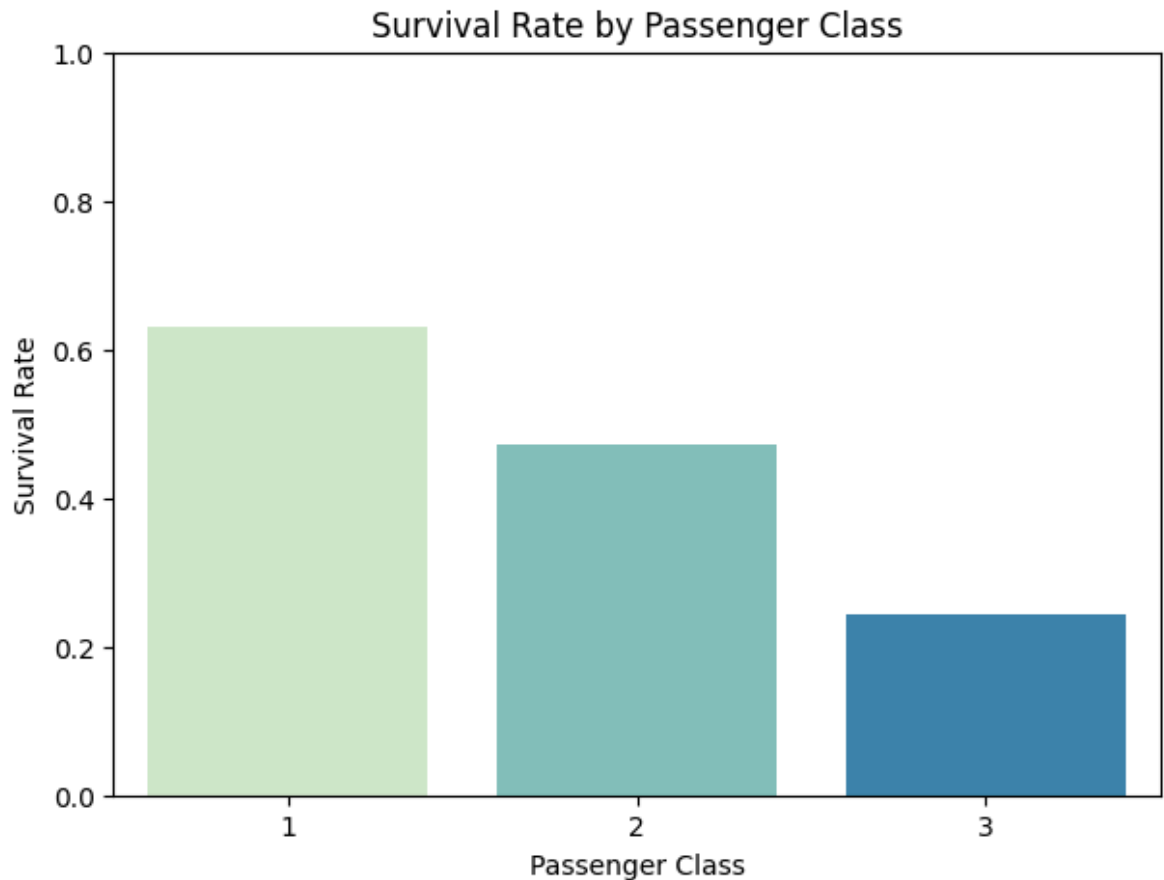
# Calculate survival rate by Pclass
pclass_survival_rate = train_df_processed.groupby('Pclass')['Survived'].mean

plt.figure(figsize=(7, 5))
sns.barplot(x='Pclass', y='Survived', data=pclass_survival_rate, palette='Gn
plt.title('Survival Rate by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Survival Rate')
plt.ylim(0, 1) # Survival rate is between 0 and 1
plt.show()
```

```
/tmp/ipykernel_33803/773412148.py:8: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Pclass', y='Survived', data=pclass_survival_rate, palette='G
```



✓ Exploring Survival Rates by Cabin Deck

The 'Cabin' feature can imply a passenger's deck, which might correlate with their proximity to lifeboats. Let's extract the deck information and analyze survival rates.

First, we need to extract the deck letter from the 'Cabin' string. If 'Cabin' is missing, we can treat it as an 'Unknown' deck.

```
# Make a copy to avoid modifying the original train_df directly
train_df_cabin_analysis = train_df.copy()

# Extract Deck from Cabin number
# The first letter of the Cabin usually indicates the deck
train_df_cabin_analysis['Deck'] = train_df_cabin_analysis['Cabin'].apply(lam

# Filter out 'T' deck (only one instance, not very informative) if present
train_df_cabin_analysis = train_df_cabin_analysis[train_df_cabin_analysis['D

# Order the decks for better visualization
deck_order = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'U']
train_df_cabin_analysis['Deck'] = pd.Categorical(train_df_cabin_analysis['De
```

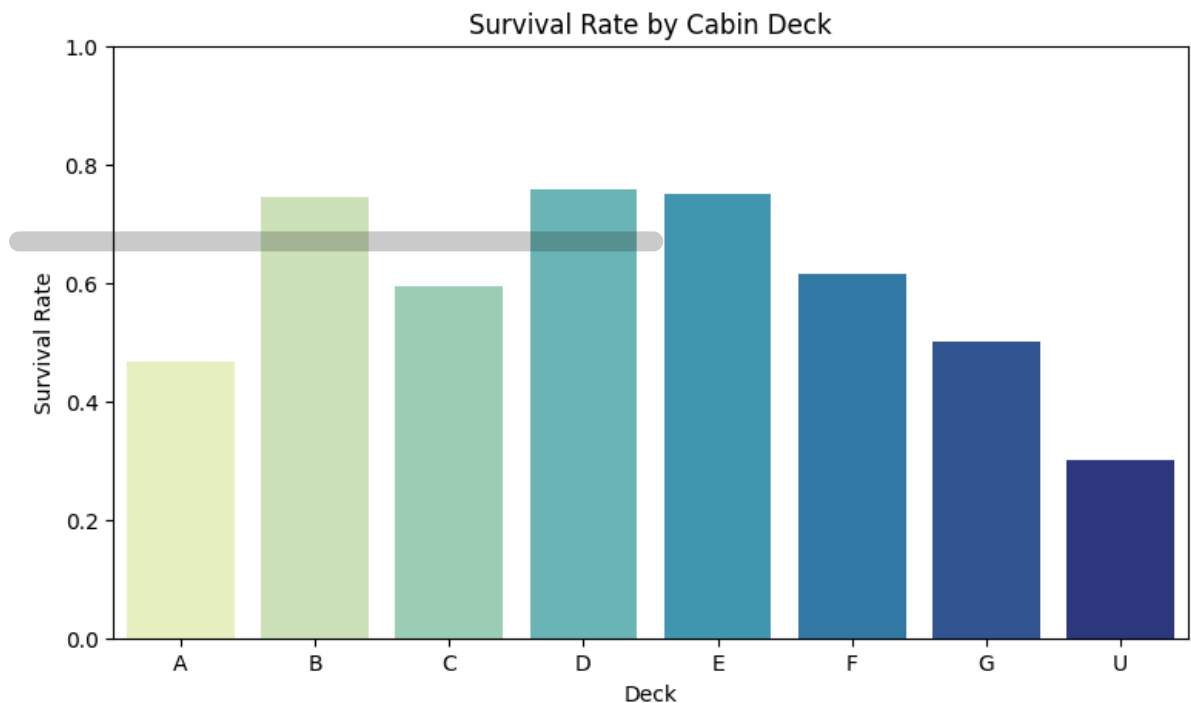
```
# Calculate survival rate by Deck
deck_survival_rate = train_df_cabin_analysis.groupby('Deck')['Survived'].mean()

plt.figure(figsize=(9, 5))
sns.barplot(x='Deck', y='Survived', data=deck_survival_rate, palette='YlGnBu')
plt.title('Survival Rate by Cabin Deck')
plt.xlabel('Deck')
plt.ylabel('Survival Rate')
plt.ylim(0, 1)
plt.show()
```

```
/tmp/ipykernel_33803/1604363795.py:16: FutureWarning: The default of observed
deck_survival_rate = train_df_cabin_analysis.groupby('Deck')['Survived'].me
/tmp/ipykernel_33803/1604363795.py:19: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in

```
sns.barplot(x='Deck', y='Survived', data=deck_survival_rate, palette='YlGnB
```



7. Conclusion and Future Work

7.1 Conclusion

This study confirms that socio-demographic features such as class, gender, and family structure are strong predictors of survival. The Gradient Boosting model demonstrated reliable performance in predicting survival outcomes.

7.2 Future Work

- Improve feature engineering for Deck variable
- Apply advanced ensemble models (e.g., XGBoost, LightGBM)
- Focus on improving recall for survival predictions

8 References

1. Titanic Dataset (train.csv and test.csv)
2. Pandas Documentation – Data Analysis
3. Seaborn & Matplotlib – Data Visualization
4. Scikit-learn – Machine Learning Models and Preprocessing

```
# 1. Install the conversion engine
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc
!pip install py pandoc

# 2. Download your notebook (.ipynb) to your local machine
# 3. Re-upload it to the folder icon on the left of Colab
# 4. Run this command (replace 'YourNotebookName.ipynb' with your file name)
!jupyter nbconvert --to pdf "titanic.ipynb"
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
Requirement already satisfied: py pandoc in /usr/local/lib/python3.12/dist-pa
[NbConvertApp] WARNING | pattern 'titanic.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.
```

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.